



Ingeniería Ejecución en Computación e informática

Estructura de datos

Clase 14: TAD Árbol

Mg. Francisco Philip Vásquez Iglesias



Resultados de aprendizaje

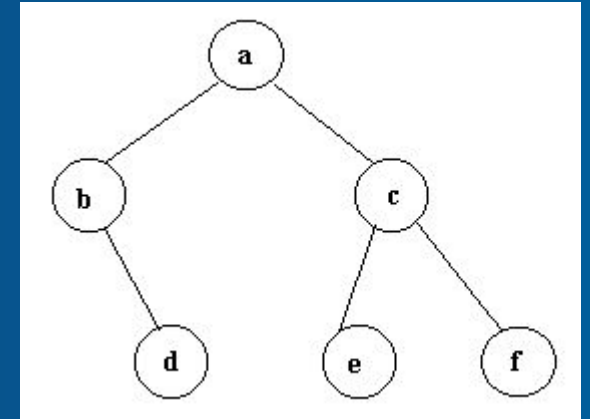
- 1.- Utilizar la notación asintótica para comparar algoritmos y determinar su calidad.
- 2.- Implementar estructuras de datos lineales para situaciones específicas en base a consideraciones de eficiencia de tiempos de acceso, utilización de recursos, tipos de implementación y complejidad.
- 3.- Implementar estructuras de datos no lineales para situaciones específicas en base a consideraciones de eficiencia de tiempos de acceso, utilización de recursos, tipos de implementación y complejidad.**

TDA Árbol

EFES

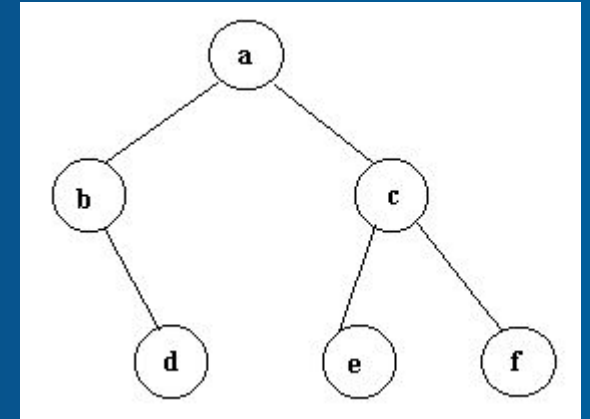
TDA Árbol

- Es una estructura de datos que puede ser definida como:
 - Una estructura vacía o...
 - Un elemento o clave de información (**nodo**) más un número finito de estructuras tipo árbol, disjuntos, llamados subárboles.



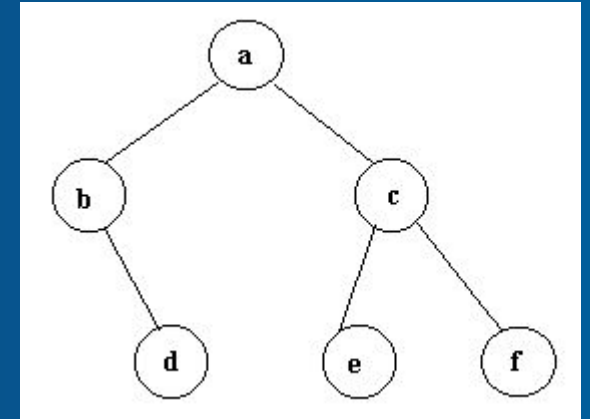
TDA Árbol

- Según el número de estructuras tipo árbol que compone a cada nodo del árbol, el árbol recibe un nombre:
 - **Binario:** el número de estructuras de cada nodo es inferior o igual a 2.
 - **N-ario:** el número de estructuras de al menos un nodo es superior a 2.
- Por lo tanto, a diferencia de la lista y sus derivados, el árbol es una estructura no secuencial.



Observación

- En la computación se utiliza mucho una estructura de datos denominada árboles binarios.
- Son árboles que tienen 0, 1 ó 2 descendientes como máximo.

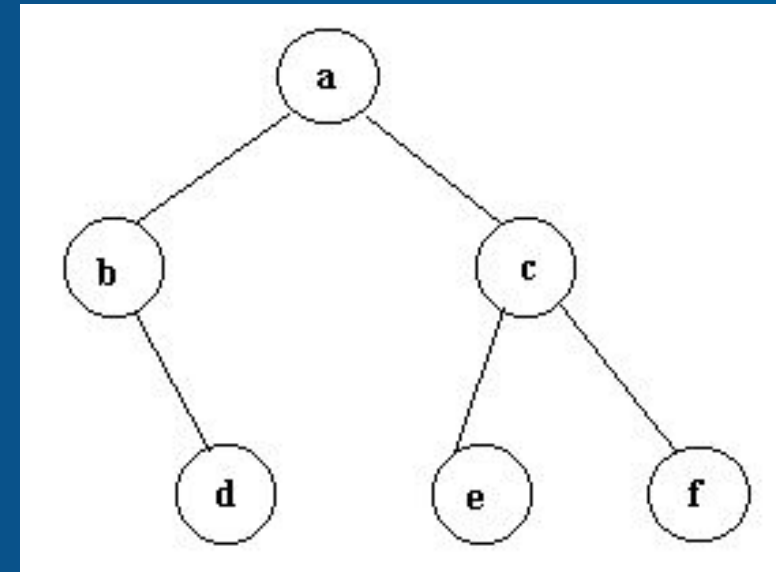


Nomenclatura de los árboles



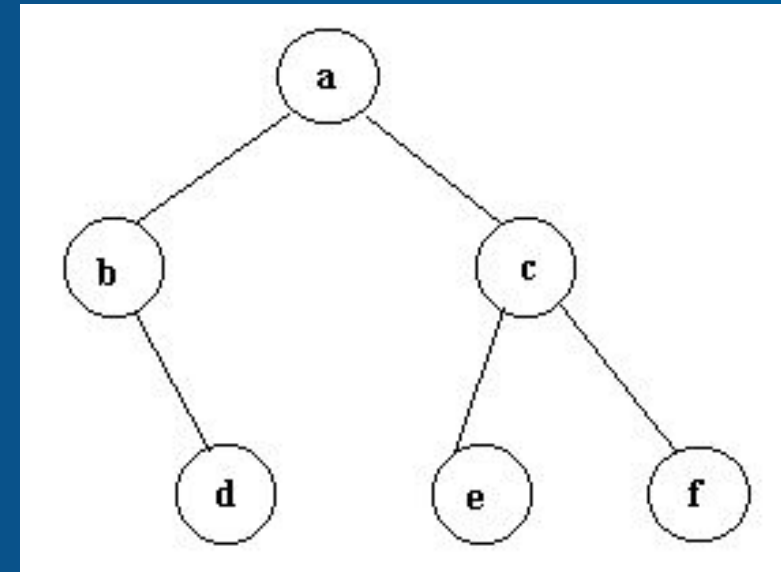
Nomenclatura de los árboles

- **Raíz:** es aquel elemento que no tiene antecesor (en la imagen de ejemplo sería el nodo “a”).
- **Rama:** arista entre dos nodos.
- **Antecesor:** un nodo X es antecesor de un nodo Y si por alguna de las ramas de X se puede llegar a Y.
- **Sucesor:** un nodo X es sucesor de un nodo Y si por alguna de las ramas de Y se puede llegar a X.



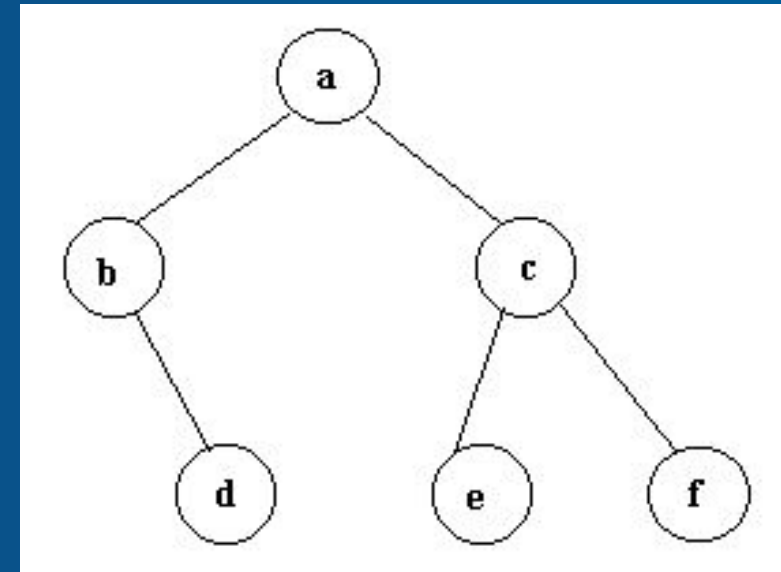
Nomenclatura de los árboles

- **Grado de un nodo:** el número de descendientes directos que tiene. Por ejemplo:
 - **c** tiene grados 2.
 - **d** tiene grado 0.
 - **a** tiene grado 2.
- **Hoja:** nodo que no tiene descendientes: grado 0. Por ejemplo:
 - **d** si es hoja.
 - **a** no es hoja.
 - **f** si es hoja.



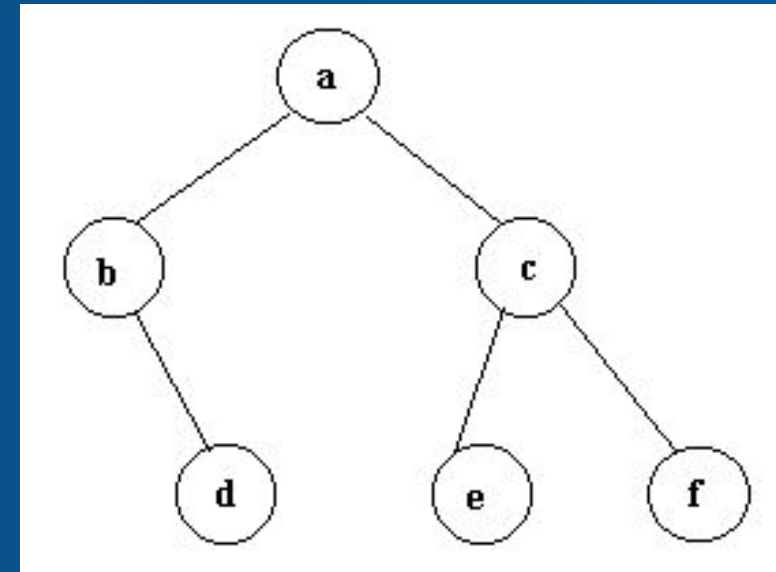
Nomenclatura de los árboles

- **Nodo interno:** aquel que tiene al menos un descendiente.
 - ***b*** si es un nodo interno.
 - ***e*** no es un nodo interno.
 - ***c*** si es un nodo interno.
- **Nivel:** número de nodos que hay que recorrer para llegar desde la raíz a un nodo. Por ejemplo:
 - El nivel del nodo ***a*** es **1**.
 - El nivel del nodo ***c*** es **2**.
 - El nivel del nodo ***e*** es **3**.



Nomenclatura de los árboles

- **Altura:** corresponde con el nivel más alto del árbol. En el ejemplo, la altura del árbol es 3.
- **Anchura:** es el mayor valor del número de nodos que hay entre cada uno de los niveles. En la figura, *la anchura es 3*.



Definición de un árbol binario en C

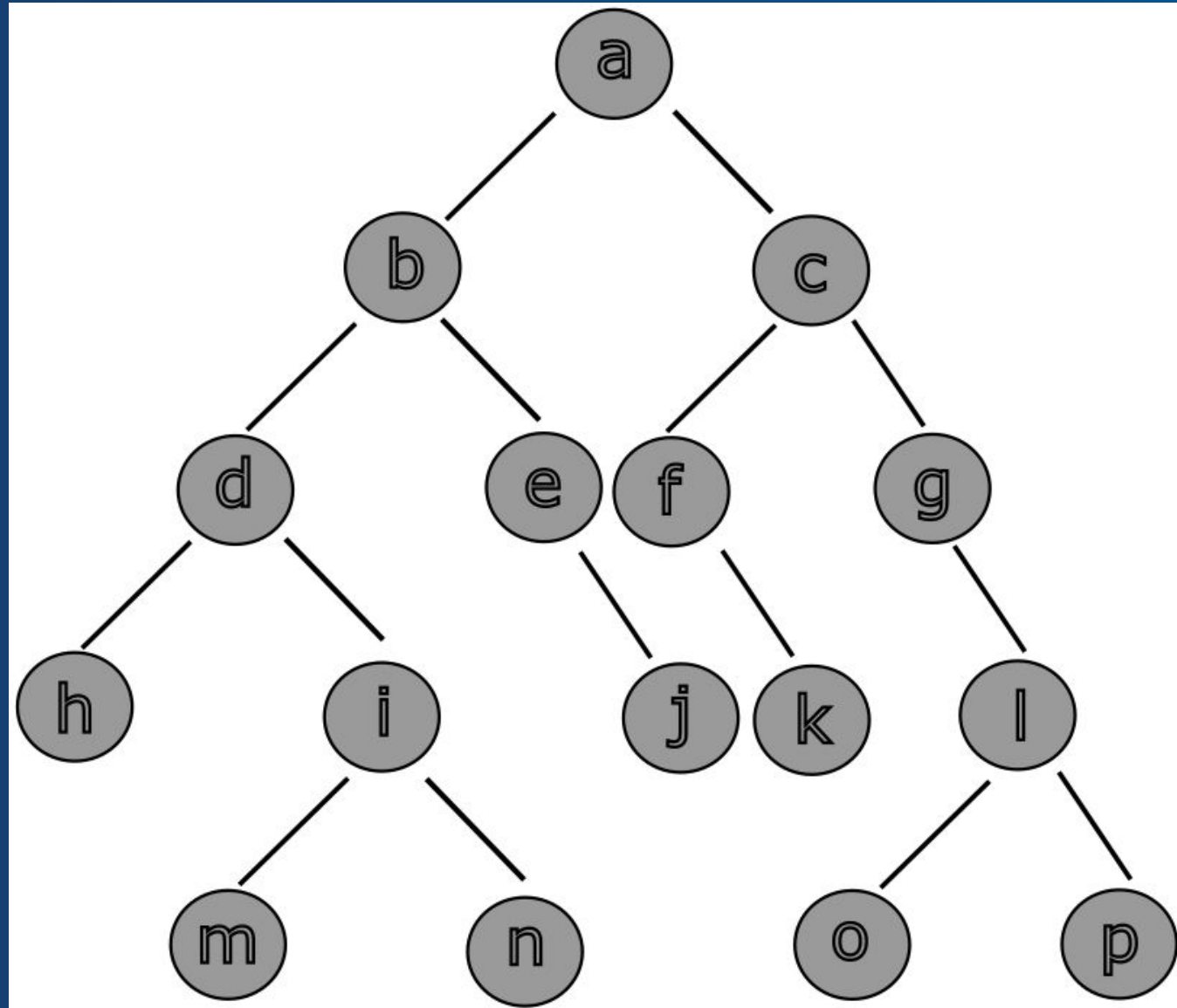


Definición de un árbol binario en C (con o sin typedef)

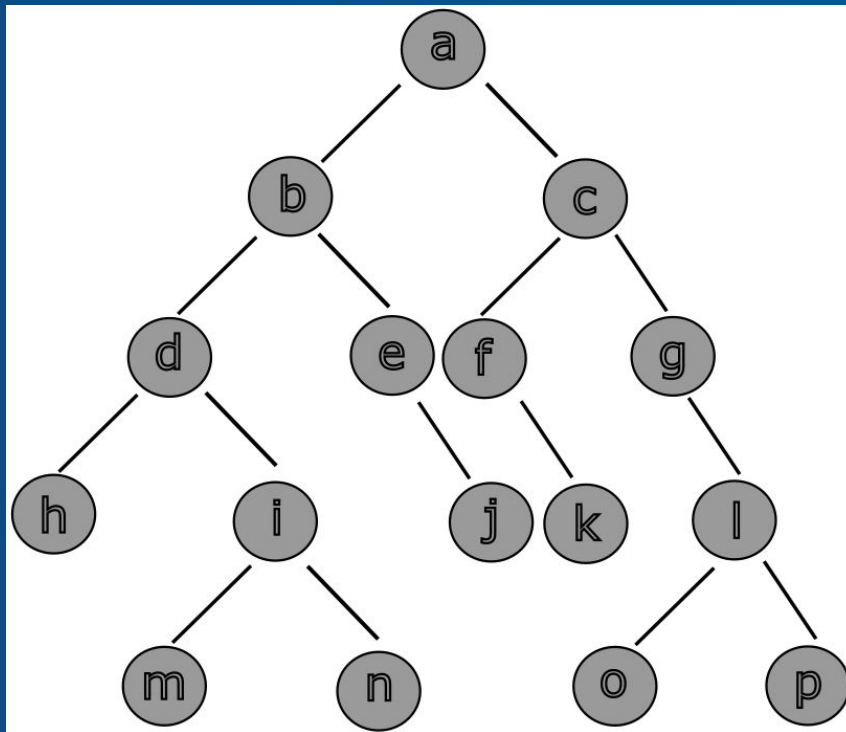
```
struct arbolBin{  
    int clave;  
    struct arbolBin *izq;  
    struct arbolBin *der;  
};
```

```
typedef struct arbolBin{  
    int clave;  
    struct arbolBin *izq;  
    struct arbolBin *der;  
}arbolBin_t;
```

Creación de un árbol binario



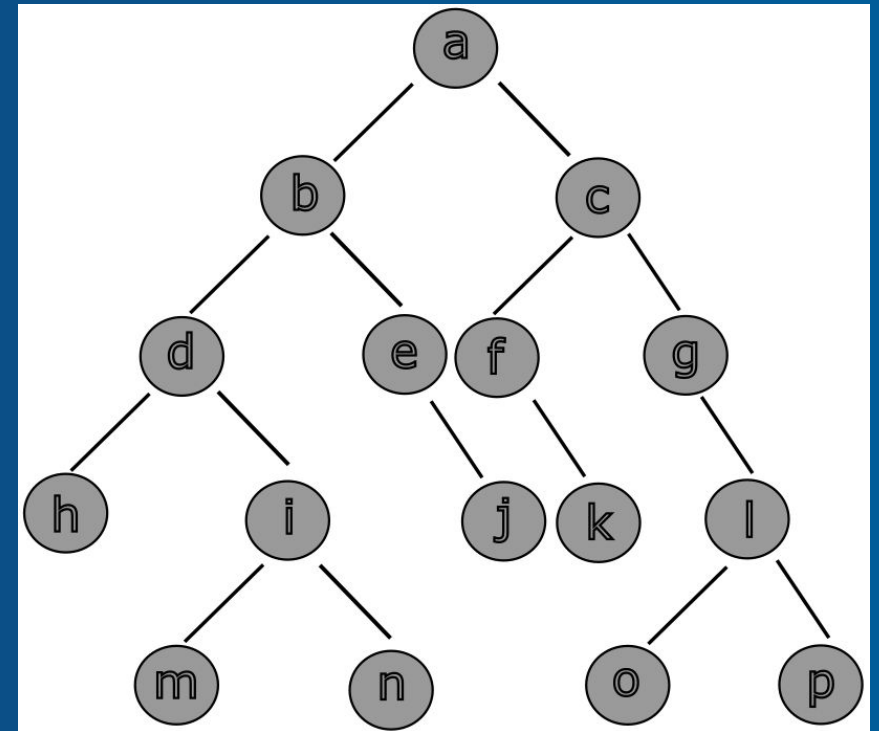
```
typedef struct arbolBin{
    char elem;
    struct arbolBin *izq;
    struct arbolBin *der;
}arbolBin_t;
```



Creación de un árbol binario

```
arbolBin_t *raiz, *a, *b, *c, *d, *e, *f, *g, *h;  
arbolBin_t *i, *j, *k, *l, *m, *n, *o, *p;
```

```
//Validar mallocs  
a=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
b=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
c=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
d=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
e=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
f=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
g=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
h=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
i=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
j=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
k=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
l=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
m=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
n=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
o=(arbolBin_t *)malloc(sizeof(arbolBin_t));  
p=(arbolBin_t *)malloc(sizeof(arbolBin_t));
```



Creación de un árbol binario

```
a->elem='a';  
a->izq=b;  
a->der=c;
```

```
b->elem='b';  
b->izq=d;  
b->der=e;
```

```
c->elem='c';  
c->izq=f;  
c->der=g;
```

```
d->elem='d';  
d->izq=h;  
d->der=i;
```

```
e->elem='e';  
e->izq=NULL;  
e->der=j;
```

```
f->elem='f';  
f->izq=NULL;  
f->der=k;
```

```
g->elem='g';  
g->izq=NULL;  
g->der=l;
```

```
h->elem='h';  
h->izq=NULL;  
h->der=NULL;
```

```
i->elem='i';  
i->izq=m;  
i->der=n;
```

```
j->elem='j';  
j->izq=NULL;  
j->der=NULL;
```

```
k->elem='k';  
k->izq=NULL;  
k->der=NULL;
```

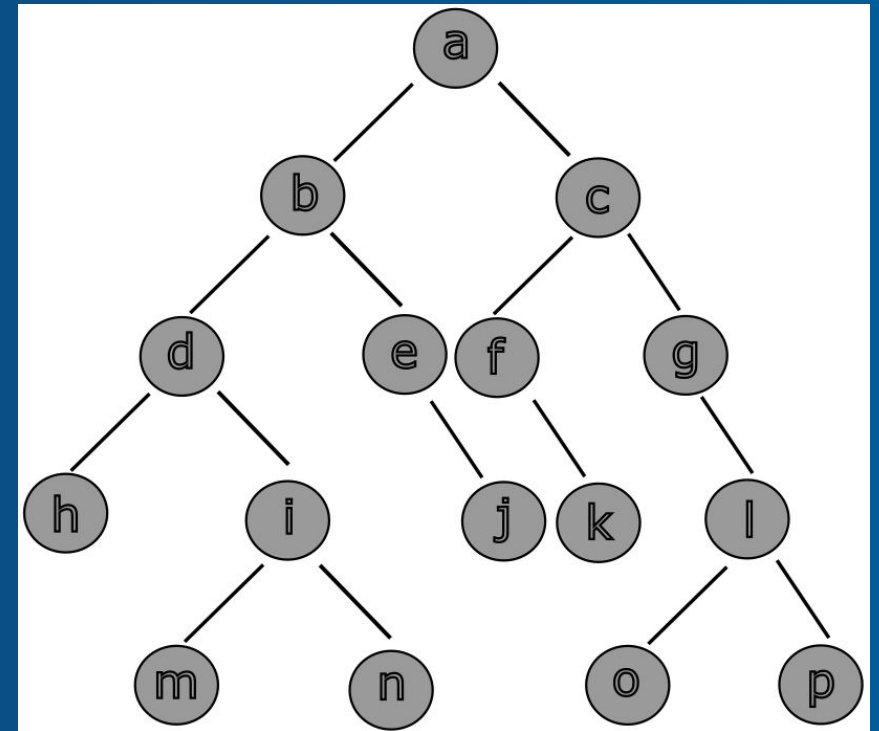
```
l->elem='l';  
l->izq=o;  
l->der=p;
```

```
m->elem='m';  
m->izq=NULL;  
m->der=NULL;
```

```
n->elem='n';  
n->izq=NULL;  
n->der=NULL;
```

```
o->elem='o';  
o->izq=NULL;  
o->der=NULL;
```

```
p->elem='p';  
p->izq=NULL;  
p->der=NULL;  
return 0;
```



Recorrido de un árbol binario

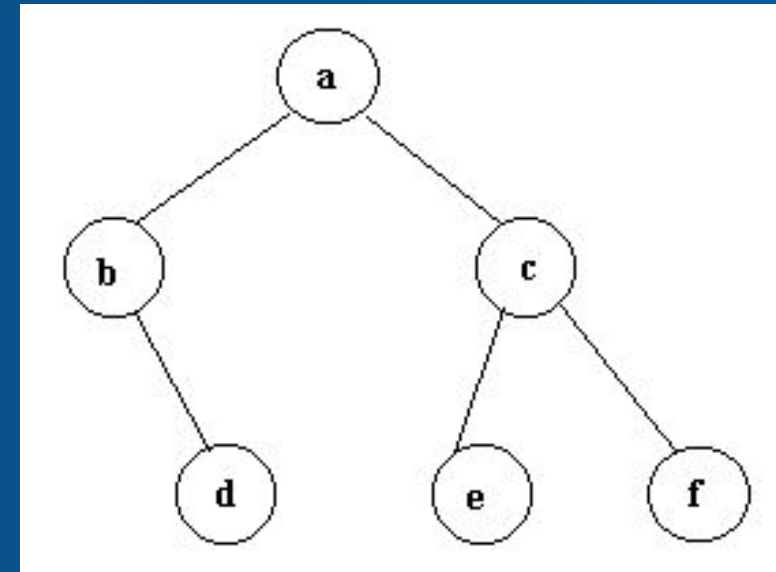


Recorrido de un árbol binario

- Existen dos tipos de recorridos:
 - Recorrido en *profundidad*.
 - Recorrido en preorden.
 - Recorrido en inorden.
 - Recorrido en postorden.
 - Recorrido en *anchura* o *a nivel*.

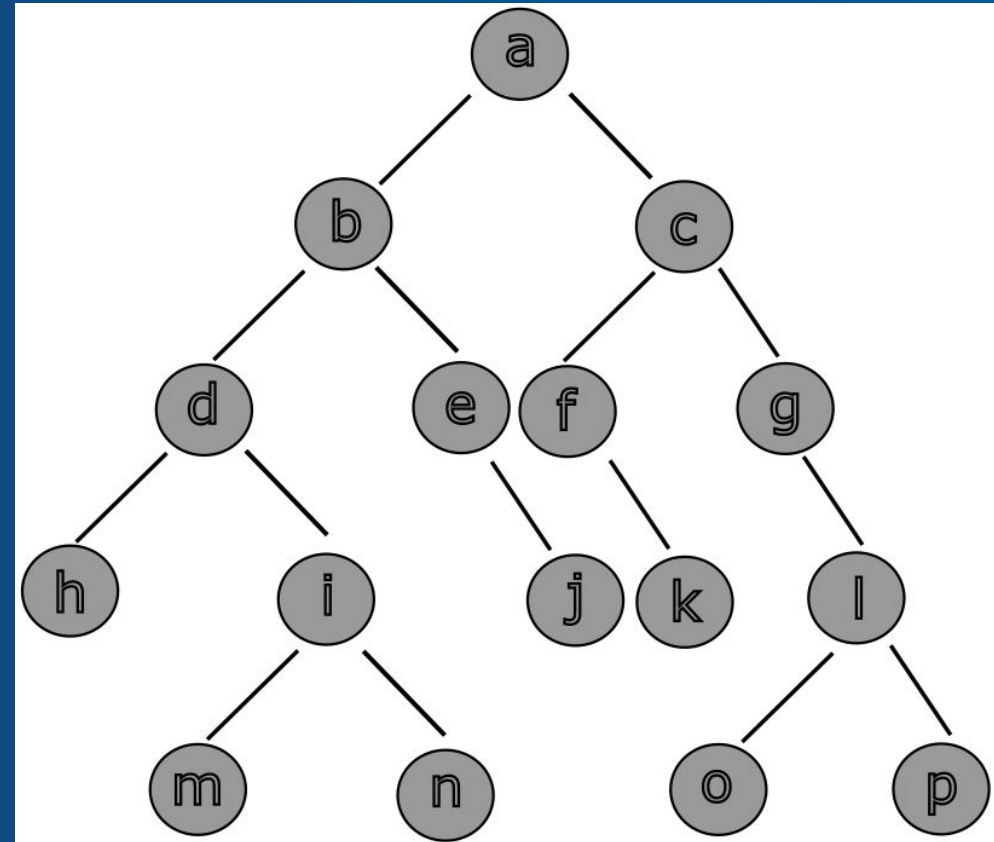
Recorrido en preorden

- Consiste en recorrer el árbol con la siguiente secuencia:
 - Visitar el nodo actual
 - Después visitar el subárbol izquierdo.
 - Después visitar el subárbol derecho.
- Es un proceso recursivo.
- El recorrido del ejemplo en preorden sería:
 - ***a, b, d, c, e y f.***

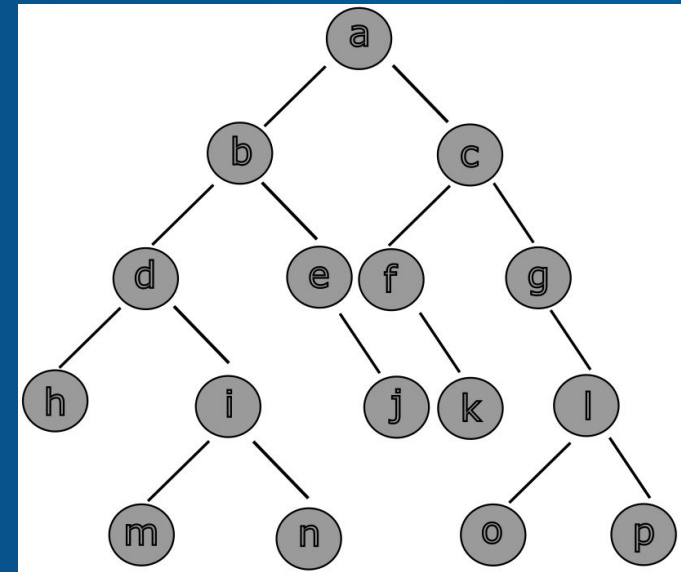


Adaptar la búsqueda en preorden a nuestro ejemplo de C

```
void preorden(arbolBin_t *a){  
    if(a!=NULL){  
        printf("%c\n",a->elem);  
        preorden(a->izq);  
        preorden(a->der);  
    }  
}
```



Adaptarlo a nuestro ejemplo de C



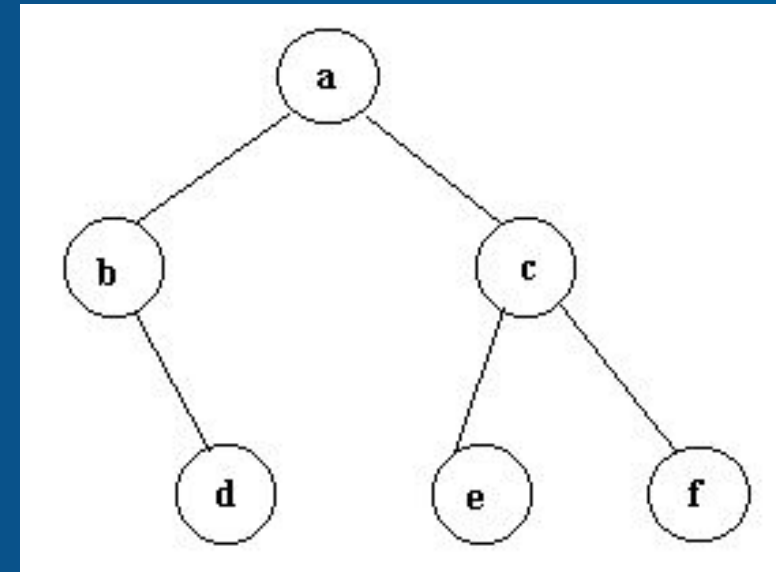
```
void preorden(arbolBin_t *a){  
    if(a!=NULL){  
        printf("%c\n",a->elem);  
        preorden(a->izq);  
        preorden(a->der);  
    }  
}
```



Se inicia el recorrido en preorden:
a b d h i m n e j c f k g l o p

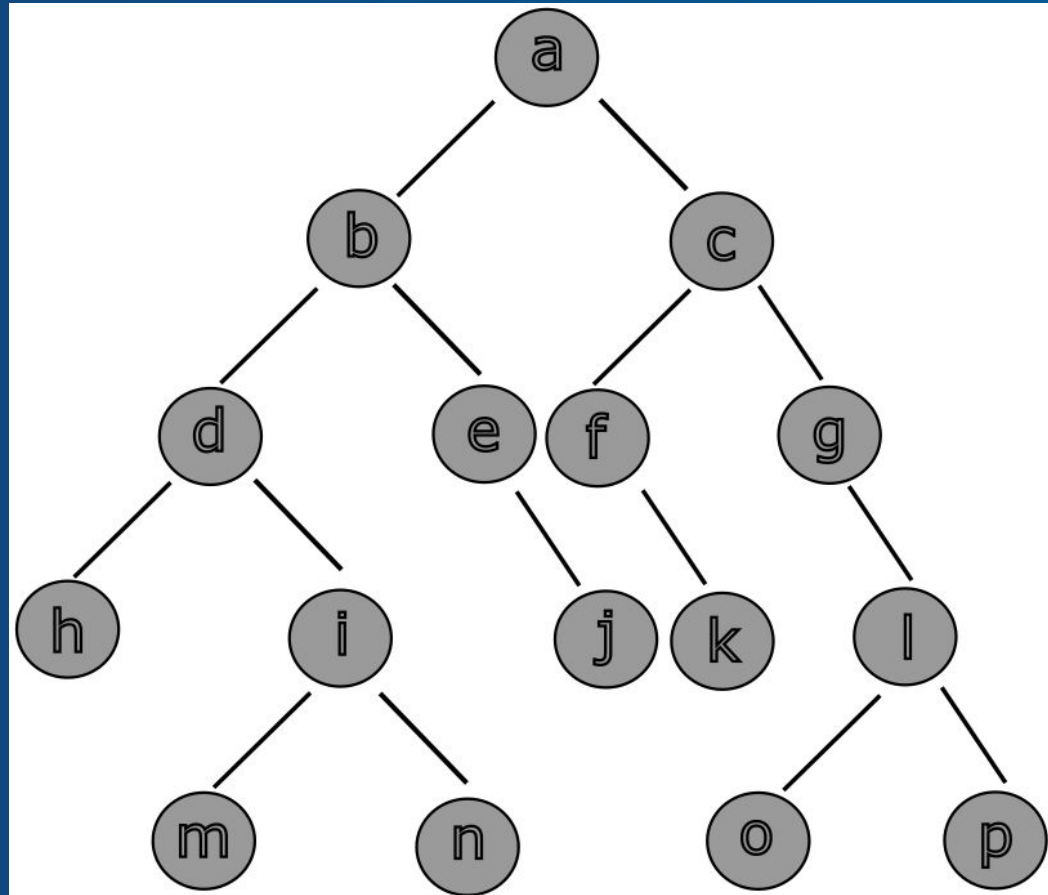
Recorrido en *inorden* u *orden central*

- Se visita en la secuencia:
 - Primero se recorre el subárbol izquierdo.
 - Luego se recorre el nodo actual.
 - Después de visita el subárbol derecho.
- En el ejemplo, el orden de las visitas serían en el orden: ***b, d, a, e, c y f.***



Adaptar la búsqueda en inorden a nuestro ejemplo de C

```
void inorden(arbolBin_t *a){  
    if(a!=NULL){  
        inorden(a->izq);  
        printf("%c ",a->elem);  
        inorden(a->der);  
    }  
}
```

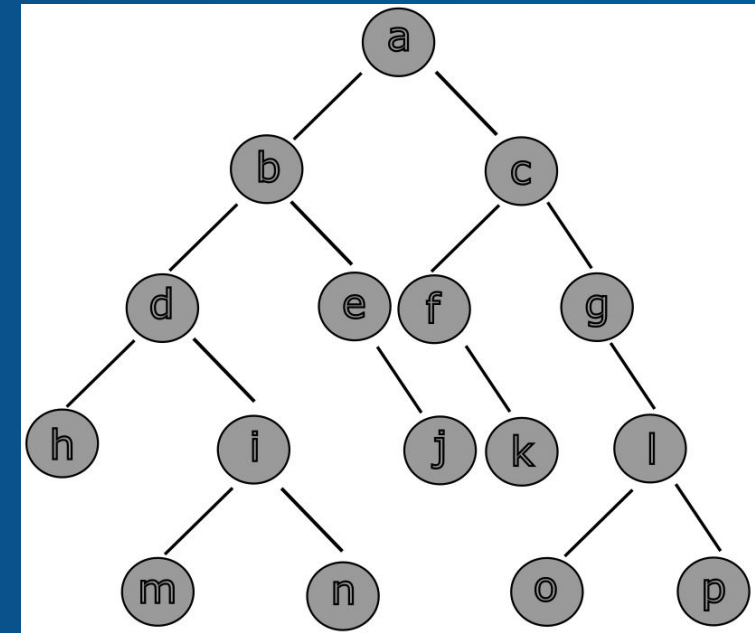


Adaptar la búsqueda en inorden a nuestro ejemplo de C

```
void inorden(arbolBin_t *a){  
    if(a!=NULL){  
        inorden(a->izq);  
        printf("%c ",a->elem);  
        inorden(a->der);  
    }  
}
```

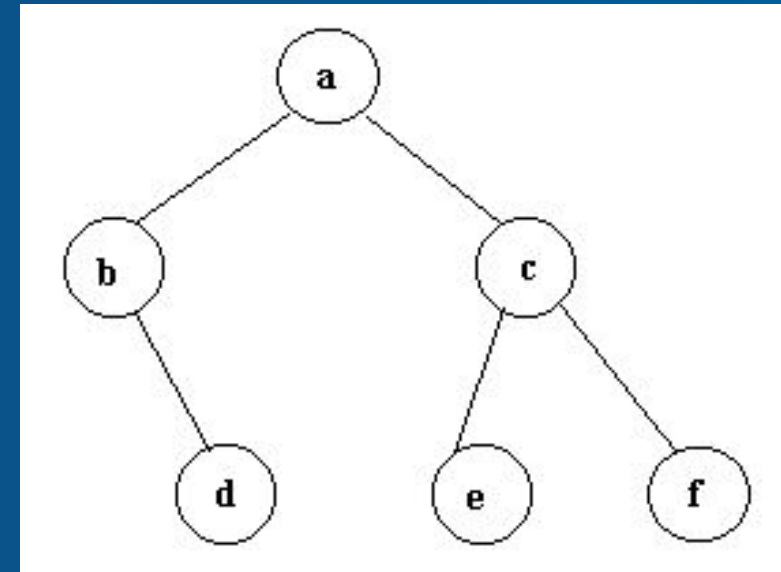


Se inicia el recorrido en inorden
h d m i n b e j a f k c g o l p



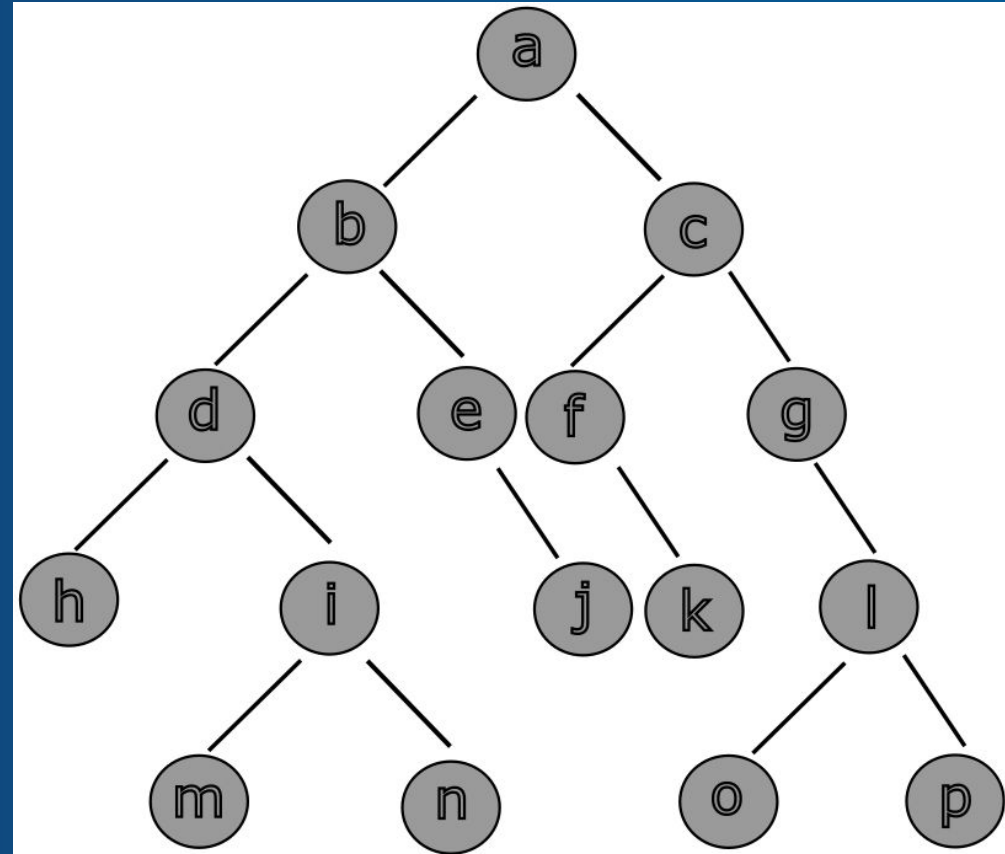
Recorrido en postorden

- Consiste en recorrer el árbol con la siguiente secuencia:
 - Primero se visita el subárbol izquierdo.
 - Luego se visita el subárbol derecho.
 - Por último se visita el nodo actual.
- En el ejemplo, el orden de las visitas serían en el orden: ***d, b, e, f, c y a.***



Adaptar la búsqueda en postorden a nuestro ejemplo de C

```
void postorden(arbolBin_t *a){  
    if(a!=NULL){  
        postorden(a->izq);  
        postorden(a->der);  
        printf("%c ",a->elem);  
    }  
}
```

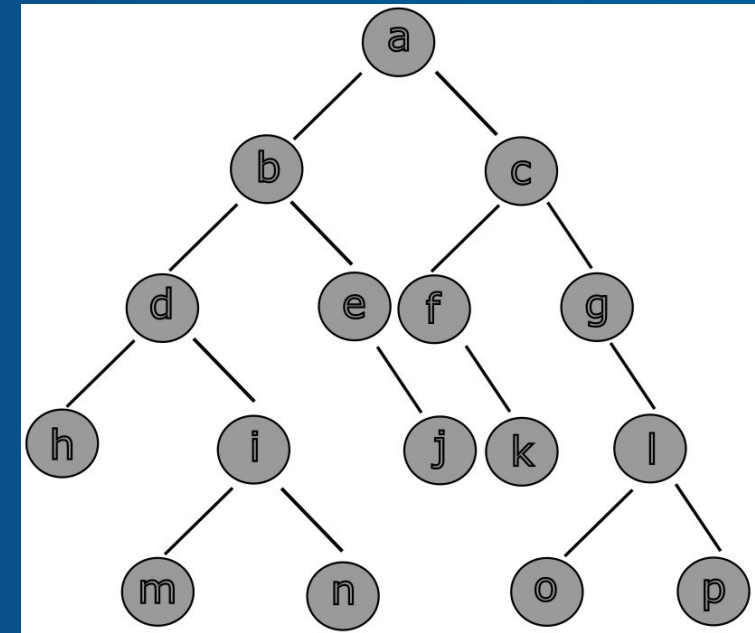


Adaptar la búsqueda en postorden a nuestro ejemplo de C

```
void postorden(arbolBin_t *a){  
    if(a!=NULL){  
        postorden(a->izq);  
        postorden(a->der);  
        printf("%c ",a->elem);  
    }  
}
```

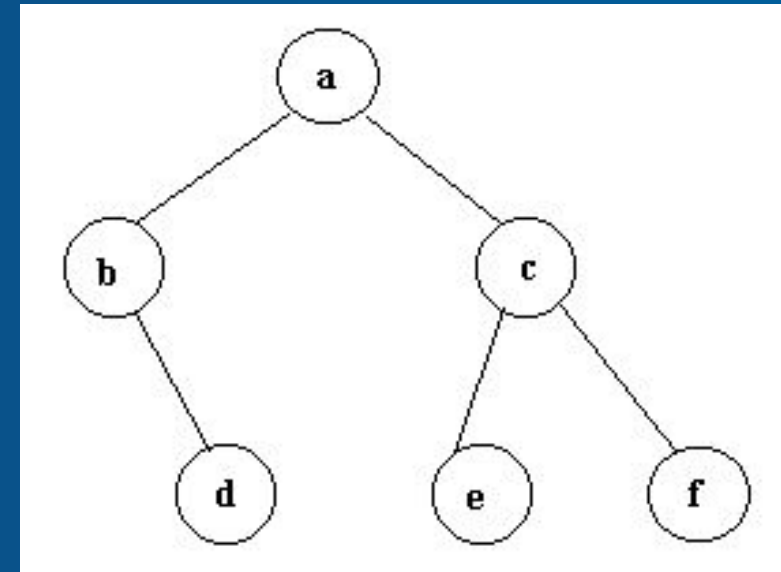


Se inicia el recorrido en postorden
h m n i d j e b k f o p l g c a



Recorrido en amplitud

- Consiste en ir visitando el árbol por niveles:
 - Primero se visitan los nodos de nivel 1 (la raíz).
 - Después los nodos de nivel 2.
 - Así hasta recorrer todo el árbol.
- Para el ejemplo, el recorrido del árbol por amplitud sería: a, b, c, d, e, f.



Recorrido en amplitud

- El recorrido en amplitud se realiza de manera iterativa, a partir de una *cola auxiliar*.

Se inicia el recorrido en amplitud
a b c d e f g h i j k l m n o p

```
void amplitud(arbolBin_t *a){
    cola_t *cola;
    arbolBin_t *aux;
    if (a != NULL){
        crear(&cola);
        encolar(&cola, a);
        while(!esVacia(cola)) {
            desencolar(&cola, &aux);
            printf("%c ", aux->elem);
            if(aux->izq != NULL){
                encolar(&cola, aux->izq);
            }
            if(aux->der != NULL){
                encolar(&cola, aux->der);
            }
        }
    }
}
```

Ejercicios



Ejercicios

- El recorrido en preorden de un determinado árbol binario es ***GEAIBMCLDFKJH*** y en inorden ***IABEGLDCFMKHJ***. Resolver:
 1. Dibujar el árbol binario.
 2. Dar el recorrido en postorden.
- Escribir una función recursiva que encuentre el número de nodos de un árbol binario.
- Escribir una función recursiva que encuentre la altura de un árbol binario.

*Anexo: Código completo
para probar*



Anexo: Código completo para probar (i)

```
2  #include<stdio.h>
3  #include<stdlib.h>
4
5  typedef struct arbolBin{
6      char elem;
7      struct arbolBin *izq;
8      struct arbolBin *der;
9  }arbolBin_t;
10
11  typedef struct tcola{
12      arbolBin_t *clave;
13      struct tcola *sig;
14  }cola_t;
```

```
16  void preorden(arbolBin_t *a);
17  void inorden(arbolBin_t *a);
18  void postorden(arbolBin_t *a);
19  void amplitud(arbolBin_t *a);
20  void crear(cola_t **cola);
21  int esVacia(cola_t *cola);
22  void encolar(cola_t **cola, arbolBin_t *elem);
23  void desencolar(cola_t **cola1, arbolBin_t **elem);
```


Anexo: Código completo para probar (ii)

```
25 int main(){
26     arbolBin_t *a, *b, *c, *d, *e, *f, *g, *h;
27     arbolBin_t *i, *j, *k, *l, *m, *n, *o, *p;
28
29     //Validar mallocs
30     a=(arbolBin_t *)malloc(sizeof(arbolBin_t));
31     b=(arbolBin_t *)malloc(sizeof(arbolBin_t));
32     c=(arbolBin_t *)malloc(sizeof(arbolBin_t));
33     d=(arbolBin_t *)malloc(sizeof(arbolBin_t));
34     e=(arbolBin_t *)malloc(sizeof(arbolBin_t));
35     f=(arbolBin_t *)malloc(sizeof(arbolBin_t));
36     g=(arbolBin_t *)malloc(sizeof(arbolBin_t));
37     h=(arbolBin_t *)malloc(sizeof(arbolBin_t));
38     i=(arbolBin_t *)malloc(sizeof(arbolBin_t));
39     j=(arbolBin_t *)malloc(sizeof(arbolBin_t));
40     k=(arbolBin_t *)malloc(sizeof(arbolBin_t));
41     l=(arbolBin_t *)malloc(sizeof(arbolBin_t));
42     m=(arbolBin_t *)malloc(sizeof(arbolBin_t));
43     n=(arbolBin_t *)malloc(sizeof(arbolBin_t));
44     o=(arbolBin_t *)malloc(sizeof(arbolBin_t));
45     p=(arbolBin_t *)malloc(sizeof(arbolBin_t));
```

```
47     a->elem='a';
48     a->izq=b;
49     a->der=c;
50
51     b->elem='b';
52     b->izq=d;
53     b->der=e;
54
55     c->elem='c';
56     c->izq=f;
57     c->der=g;
58
59     d->elem='d';
60     d->izq=h;
61     d->der=i;
62
63     e->elem='e';
64     e->izq=NULL;
65     e->der=j;
66
67     f->elem='f';
68     f->izq=NULL;
69     f->der=k;
```

```
71     g->elem='g';
72     g->izq=NULL;
73     g->der=l;
74
75     h->elem='h';
76     h->izq=NULL;
77     h->der=NULL;
78
79     i->elem='i';
80     i->izq=m;
81     i->der=n;
82
83     j->elem='j';
84     j->izq=NULL;
85     j->der=NULL;
86
87     k->elem='k';
88     k->izq=NULL;
89     k->der=NULL;
90
91     l->elem='l';
92     l->izq=o;
93     l->der=p;
```

```
95     m->elem='m';
96     m->izq=NULL;
97     m->der=NULL;
98
99     n->elem='n';
100    n->izq=NULL;
101    n->der=NULL;
102
103    o->elem='o';
104    o->izq=NULL;
105    o->der=NULL;
106
107    p->elem='p';
108    p->izq=NULL;
109    p->der=NULL;
```

Anexo: Código completo para probar (iii)

```
111     printf("\nSe inicia el recorrido en preorden:\n");
112     preorden(a);
113     printf("\nSe inicia el recorrido en inorden\n");
114     inorden(a);
115     printf("\nSe inicia el recorrido en postorden\n");
116     postorden(a);
117     printf("\nSe inicia el recorrido en amplitud\n");
118     amplitud(a);
119
120     return 0;
121 }
```

```
123 void preorden(arbolBin_t *a){
124     if(a!=NULL){
125         printf("%c ",a->elem);
126         preorden(a->izq);
127         preorden(a->der);
128     }
129 }
```

```
131 void inorden(arbolBin_t *a){
132     if(a!=NULL){
133         inorden(a->izq);
134         printf("%c ",a->elem);
135         inorden(a->der);
136     }
137 }
```

```
139 void postorden(arbolBin_t *a){
140     if(a!=NULL){
141         postorden(a->izq);
142         postorden(a->der);
143         printf("%c ",a->elem);
144     }
145 }
```


Anexo: Código completo para probar (iv)

```
147 void amplitud(arbolBin_t *a){
148     cola_t *cola;
149     arbolBin_t *aux;
150     if (a != NULL){
151         crear(&cola);
152         encolar(&cola, a);
153         while(!esVacia(cola)) {
154             desencolar(&cola, &aux);
155             printf("%c ", aux->elem);
156             if(aux->izq != NULL){
157                 encolar(&cola, aux->izq);
158             }
159             if(aux->der != NULL){
160                 encolar(&cola, aux->der);
161             }
162         }
163     }
164 }
```

```
166 //Cola circular de arboles binarios
167 void crear(cola_t **cola){
168     *cola = NULL;
169 }
170
171 int esVacia(cola_t *cola){
172     return (cola == NULL);
173 }
```


Anexo: Código completo para probar (v)

```
175 void encolar cola_t **cola, arbolBin_t *elem){
176     cola_t *nuevo;
177     nuevo = (cola_t *) malloc(sizeof(cola_t));
178     nuevo->clave = elem;
179     nuevo->sig = NULL;
180
181     if (*cola == NULL){
182         *cola = nuevo;
183     }
184     else{
185         cola_t *temp = *cola;
186         while (temp->sig != NULL) {
187             temp = temp->sig;
188         }
189         temp->sig = nuevo;
190     }
191 }
```

```
193 void desencolar cola_t **cola1, arbolBin_t **elem){
194     cola_t *aux;
195     *elem = (*cola1)->clave;
196     aux = *cola1;
197     *cola1 = (*cola1)->sig;
198     free(aux);
199 }
```



Ingeniería Civil informática

Estructura de datos

Fin de la clase

Mg. Francisco Philip Vásquez Iglesias

